TabGis - Runbook de Execucao Autonoma

Objetivo

Executar uma janela longa de trabalho automatizado com checkpoints, sem interromper em falhas pontuais.

Script principal

- `scripts/autonomy/run_10h.sh`
- `scripts/autonomy/run_window_10h.sh` (orquestrador por janela de 10 horas)

Como iniciar

Da raiz do repositorio:

```
bash scripts/autonomy/run_10h.sh
```

Para janela continua de 10h (ciclos repetidos):

```
bash scripts/autonomy/run_window_10h.sh
```

Até às **07:00** do relógio local (próxima ocorrência):

```
bash scripts/autonomy/run_until_0700.sh
```

Onde ficam os artefatos

- Pasta: `autonomy-logs/run_YYYYMMDD_HHMMSS/`
- Arquivos principais:
  - `timeline.log`
  - `status_table.md`
  - `final_report.md`
  - `*.log` por etapa

Comportamento

- O pipeline e dividido em ondas.
- Cada etapa registra comando, status e exit code.
- Em falha, a execucao continua para nao perder a janela de autonomia.

Stress API (manual ou CI)

Script: `scripts/stress/stress_api.sh`

```
STRESS_BASE_URL=http://127.0.0.1:8088 STRESS_ITERATIONS=200 bash scripts/stress/stress_api.sh
```

Gera JSON em `autonomy-logs/stress_*.json` (ou defina `STRESS_REPORT_FILE`). O passo **2.4** de `run_10h.sh` executa uma carga leve por defeito (`STRESS_ITERATIONS=50`), usando `PLAYWRIGHT_BASE_URL` quando definida.

`STRESS_CONCURRENCY` (default 1): acima de 1, cada iteração dispara ondas de pedidos paralelos (round-robin nos quatro endpoints).

Observacoes operacionais

- Se `npm` nao estiver instalado, as etapas de E2E sao marcadas como falha e o fluxo segue.
- O report final centraliza a trilha completa para auditoria de execucao.